

target: 总结一些基本算法之外的巧思和错误经验

错误算法

剔除字符串中字符

```
// L1-011 A-B
// 2024年12月8日11:31:13
// 2024年12月8日12:13:41
// 部分正确方法
int main() {
    string input, compair, result;
    getline(cin, input);
    getline(cin, compair);
    for (int i = 0; i < compair.length(); i++) {
        for (int j = 0; j < input.length(); j++) {
            if (compair[i] == input[j]) {
                input.erase(input.begin() + j);
            }
        }
    }
    cout << input;
}
```

错误原因：

一旦 if 匹配，删除该位置（下标为 index）的字符，该位置的下一个字符的补上并且下标变为 index，下次循环从 j++（也就是 index+1）开始会导致漏掉一个。所以这个算法不能处理有连续的 compair 中字符的字符，如字符“aaaaa”，剔除“a”，返回结果为“aaa”

备注：

即使将 `input.erase(input.begin() + j)` 改为 `input.erase(input.begin() + j - 1)` 当 `j = input.length()` 会漏掉最后一个重复元素，同样错误

教训：

- 再循环中使用动态数组时，如果需要控制次数跟数组大小有关，最好只操作数组第一个数或者最后一个数字，这样可以防止下标变化导致漏掉容器中元素
- 需要比对的问题，可以试着在一个循环中初始化两个变量控制两个比较对象而不要用两个循环徒增复杂度，但需要额外的容器存储

将字符串中单个字符串转换为数字

```
string s;
for(char c : s){
    return static_cast<int>(c)
}
```

string 类型[[用法导向知识#各种 STL 容器底层实现和操作复杂度|底层由动态数组实现]]，单个元素是 `char` 类型，可以直接使用 `int number = s[0] - '0'` 得到字符的大小，`static_cast<int>(c)` 将 `char` 转化为 `int` 是向上转换，返回的是 `c` 的 ASCII 编号而不是真正的数值

过于复杂的思维

思考不全面的模拟做法

3152. 特殊数组 II

(<https://leetcode.cn/problems/special-array-ii/>)

做了一个小时还是错了 95% 以上测试数据通过，忽略哈希表中没查找的情况

```
class Solution {
public:
    vector<bool> isArraySpecial(vector<int>& nums, vector<vector<int>>& queries) {
        int n = nums.size(), q_size = queries.size();
        for(int& x : nums){
            // mark1
            x = x % 2;
        }
        vector<bool> ans(queries.size());
        unordered_map<int, int> mapping;
        int first = 0, last = 0;
        for (int i = 1; i < n; i++){
            //mark2
            if(nums[i - 1] % 2 == nums[i] % 2){
                mapping.insert({first, i - 1});
                first = i;
            }
            i++;
        }
        mapping.insert({first, nums.size() - 1});
    }
};
```

```

        for (int i = 0; i < q_size;i++){    // mark3
            if(queries[i][0]==queries[i][1]){
                ans[i] = true;
                continue;
            }
            auto it = mapping.find(queries[i][0]); //mark4
            (it==mapping.end()||it->second<queries[i][1]) ? ans[i] = false : ans[i] = true;
        }
        return ans;
    }
};

```

思路是正确的：

- 只关心元素奇偶性，所以数组中只需要存在 0 和 1 (mark 1)
 - 记录每一个满足奇偶交替的子数组的**起始（键值）和结束位置**，(mark 2)
 - 逐个判断 queries 中元素 (mark 3)
 - 没有查找到有两种情况：(mark 4)
1. `queries[i][0]` 位于某个记录的键值对 `pair{a, b}` 之间，也就是 `a<i<b`，应该插入 `true`
 2. `queries[i][0]` 没有记录，应该插入 `false`

以上算法没有考虑到第一种情况，并且也很难实现第一种方法

改用库函数方法：(`adjacent_find`)

```

class Solution {
public:
    vector<bool> isArraySpecial(vector<int>& nums, vector<vector<int>>& queries) {
        int n = nums.size(), m = queries.size();
        vector<bool> ans(m);
        for (int i = 0; i < m;i++){
            auto it = adjacent_find(nums.begin() + queries[i][0], nums.begin() + queries[i][1],
                { return ((a ^ b) & 1) == 0; });
            (it==nums.begin()+queries[i][1]+1) ? ans[i] = true; : ans[i] = false;
        }
        return ans;
    }
};

```

- `adjacent_find` 方法，在范围 `[first, last)` 中搜索两个连续的相等元素。通过定义二元操作来寻找指定的相邻元素
 - `op` 定义寻找两个相邻元素，他们的 `(a ^ b) & 1) == 0`，即奇偶相同
 - 需要注意 `last` 是开区间，从外部传入迭代器变量时，需要 `+1`
 - `adjacent_find` 只会寻找范围内第一个满足的结果，和其他 `find` 函数一致
 - 返回指向首对满足 `op` 操作的首个元素的迭代器，即 `*it == *(it + 1)`
 - 复杂度为 `O(n)`
- 总时间为 `O(n^2)` 超时
前缀和思想：

```
class Solution {
public:
    vector<bool> isArraySpecial(vector<int>& nums, vector<vector<int>>& queries) {
        int n = nums.size(), q = queries.size();
        vector<int> s(n);
        vector<bool> ans(q);
        for (int i = 1; i < n; i++) {
            s[i] = s[i - 1] + ((nums[i] ^ nums[i - 1] ^ 1) & 1);
        }
        for (int i = 0; i < q; i++) {
            ans[i] = s[queries[i][0]] == s[queries[i][1]];
        }
        return ans;
    }
};
```

- 将数组分为符合奇偶相间和不符合奇偶相间的不同分段
 - 如果一个元素和前一个元素奇偶不同，设置为 0，反之设置为 1
 - `s` 存储前缀和，这样可以计算任何一个子数组的元素和，元素和 `>0` 表示数组中有奇偶不相间的部分，`==0` 表示数组奇偶相间符合要求
 - 进一步化简为 `s[i] != s[j]` 时奇偶不相间，反之奇偶相间
- 再从 `queries` 中遍历数据从 `s` 中查询即可 `a`

迭代器，下标的错误

2055. 蜡烛之间的盘子

(<https://leetcode.cn/problems/plates-between-candles/>)

这是错误的解法

```
class Solution {
public:
    vector<int> platesBetweenCandles(string s, vector<vector<int>>& queries) {
        int n = s.length();
        vector<int> str(n), ans(queries.size());
        vector<int> pre_sum(n + 1);
        for (int i = 0; i < n; i++) {
            s[i] == '*' ? str[i] = 0 : str[i] = 1;
            pre_sum[i + 1] = pre_sum[i] + str[i];
        }
        for (auto x : queries) {
            auto left1 = str.begin() + x[0], right1 = str.begin() + x[1];
            auto it1 = find_if(left1, right1, 1);
            auto left2 = reverse_iterator<decltype(left1)>(left1);
            auto right2 = reverse_iterator<decltype(right1)>(right1);
            auto it2 = find_if(right2, left2, 1);
            int length = x[1] - x[0] + 1, first = 0, second = 0;
            if (it1 != str.end() && it2 != str.rend()) {
                first = it1 - str.begin(), second = length - (it2 - str.rbegin());
                ans.emplace_back(length - (pre_sum[second + 1] - pre_sum[first]));
            }
        }
        return ans;
    }
};
```

- 思路正确，先找到需要查询的位置，得到位置长度-对应前缀和得到盘子数量
 - 但找到范围内的第一个蜡烛和**最后一个蜡烛**，在找最后一个时很容易因迭代器有方向，下标没有方向而弄错，参考[[用法导向知识#迭代器的转化|正反向迭代器相互转化]]
- 正确做法：

```
class Solution {
public:
    vector<int> platesBetweenCandles(string s, vector<vector<int>>& qs) {
        vector<int> ans(qs.size(), 0);
```

```

vector<int> sum(s.length() + 1, 0);
vector<int> list;
for (int i = 0; i < s.length(); i++) {
    if (s[i] == '|') list.push_back(i);
    sum[i + 1] = sum[i] + (s[i] == '*' ? 1 : 0);
}
if (list.empty()) return ans;
for (int i = 0; i < qs.size(); i++) {
    int a = qs[i][0], b = qs[i][1];
    int c = -1, d = -1;
    // 找到 a 右边最近的蜡烛
    int l = 0, r = list.size() - 1;
    while (l < r) {
        int mid = l + r >> 1;
        if (list[mid] >= a) r = mid;
        else l = mid + 1;
    }
    if (list[r] >= a) c = list[r];
    else continue;
    // 找到 b 左边最近的蜡烛
    l = 0; r = list.size() - 1;
    while (l < r) {
        int mid = l + r + 1 >> 1;
        if (list[mid] <= b) l = mid;
        else r = mid - 1;
    }
    if (list[r] <= b) d = list[r];
    else continue;
    if (c <= d) ans[i] = sum[d + 1] - sum[c];
}
return ans;
}
};

```

在这一题上，可以不需要前缀和，因元素是二元的

```

class Solution {
public:

```

```

vector<int> platesBetweenCandles(string s, vector<vector<int>>& queries) {
    vector<int> list, ans;
    for (int i = 0; i < s.length(); i++)
        if (s[i] == '|')
            list.emplace_back(i);
    if (list.size() <= 1) return {0};
    for (auto x : queries) {
        auto left = lower_bound(list.begin(), list.end(), x[0]);
        auto right = prev(upper_bound(list.begin(), list.end(), x[1]));
        if (left >= right) {
            ans.emplace_back(0);
        } else {
            ans.emplace_back(*right - *left - distance(left, right));
        }
    }
    return ans;
}
};

```

- 记下每一个蜡烛出现的位置，再查询时，`distance(right-left)` = 查询范围内有几根蜡烛，因为 `list` 中只存储了蜡烛位置
- `*right - *left = 查询范围长度`，两者之差得到盘子数量
还可以使用预处理，将算法题变为查询题

```

class Solution {
public:
    vector<int> platesBetweenCandles(string s, vector<vector<int>>& qs) {
        vector<int> l(s.length(), 0), r(s.length(), 0);
        vector<int> pre_sum(s.length() + 1, 0);
        for (int i = 0, j = s.length() - 1, closest_left = -1, closest_right = -1; i < s.length(); i++)
            if (s[i] == '|') closest_left = i;
            if (s[j] == '|') closest_right = j;
            l[i] = closest_left; r[j] = closest_right;
            pre_sum[i + 1] = pre_sum[i] + (s[i] == '*' ? 1 : 0);
        }
        vector<int> ans(qs.size(), 0);
        for (int i = 0; i < qs.size(); i++) {
            int a = qs[i][0], b = qs[i][1];

```

```

        int c = r[a], d = l[b];

        if (c != -1 && c <= d) ans[i] = pre_sum[d + 1] - pre_sum[c];
    }

    return ans;
}

};

```

- 记下每一个位置（无论是盘子还是蜡烛）左右两边最近的（包括自己如果是蜡烛的话）的蜡烛和前缀和
- 需要时直接查询出左右边界，然后用前缀和求出盘子数量

没有用程序思维思考

PTA L1-018 大笨钟

反面教材

```

pair<int,int> get_time(string s){
    int hour = stoi(s.substr(0, 2));
    int minute = stoi(s.substr(3, 2));
    return make_pair(hour, minute);
}

bool is_too_early(pair<int,int> time){
    return time.first < 12||(time.first==12 && time.second!=0);
}

int main(){
    string s ;
    cin >> s;

    pair<int, int> time = get_time(s);
    int count = time.first;
    if(!is_too_early(time)&&time.second != 0){
        count++;
    }

    if(is_too_early(time)){
        cout << "Only " << s << ".  Too early to Dang.";
    }else{
        for (int i = 0; i < count % 12;i++){
            cout << "Dang";
        }
    }
}

```

```
}  
  
}
```

is_too_early 函数逻辑比较复杂，判定逻辑也要依靠 pair 来判断，实际上 `hh:mm` 时间完全可以转化为分钟，只用分钟作为判断依据

```
int get_time(string s){  
    int hour = stoi(s.substr(0, 2));  
    int minute = stoi(s.substr(3, 2));  
    return hour * 60 + minute;  
}  
  
int main(){  
    string s ;  
    cin >> s;  
    int result = get_time(s);  
    int count = result / 60 - 12;  
    if(result<=720){  
        cout << "Only " << s << ". Too early to Dang.";  
    }else if(result % 60 != 0){  
        count++;  
    }  
    for (int i = 0; i < count;i++){  
        cout << "Dang";  
    }  
}
```

需要简化重复过程

各个步骤操作交叉部分合并

461. 汉明距离

(<https://leetcode.cn/problems/hamming-distance/>)

优化前：

```
class Solution {  
public:  
    vector<int> to_binary(int x){  
        vector<int> vec = {};
```

```

        while(x){
            vec.push_back(x % 2);
            x /= 2;
        }
        return vec;
    }

    int hammingDistance(int x, int y) {
        vector<int> vec1 = to_binary(x);
        vector<int> vec2 = to_binary(y);
        int difference = vec2.size() - vec1.size();
        if(difference < 0){
            difference = -difference;
            for (int i = 0; i < difference;i++){
                vec2.push_back(0);
            }
        }else{
            for (int i = 0; i < difference;i++){
                vec1.push_back(0);
            }
        }

        int count = 0;
        for (int i = 0; i < vec2.size();i++){
            if(vec1[i]+vec2[i]==1){
                count++;
            }
        }
        return count;
    }
};

```

错误分析：

- 题目怎么说就怎么写，首先将两个数字转化为二进制，然后填充 0，最后异或比较
- 需要单独写一个判断函数，并且各种额外容器存储

优化方向：

- 在循环中比较，不用额外创建两个容器存储转化二进制后的解法
- 转二进制方法是除二取余法——两个数需要两个循环
- 取余后**对两个数比较**，所以可以将两个取余放在一个循环中，依次循环中取两个数的余并比较
- 填充的数字是 0，0/2 也等于 0，所以不用补位

```

while(x!=0||y!=0){ // 两个循环合并为一个
    if(x%2 == y%2){ // 取余判断合并为一个
        count++;
    }
    x /= 2;
    y /= 2;
}

```

解法：

```

class Solution {
public:
    int hammingDistance(int x, int y) {
        int count = 0;
        if(x==y){
            return 0;
        }
        while (x != 0||y!=0){
            if(x%2!=y%2){
                count++;
            }
            x /= 2;
            y /= 2;
        }
        return count;
    }
};

```

原地求和问题

搞清楚题目需要什么比较，什么内容是需要运算过程中留下来的，什么是不需要的，能够原地更新就不要创造新的容器存储冗余数据（复制和构造新元素很费时间）

2815. 数组中最大数对和

(<https://leetcode.cn/problems/max-pair-sum-in-an-array/>)

优化前：

```

class Solution {
public:
    // 获取数字的最高位
    int findMaxDigit(int number) {
        if (number == 0) return 0; // 0 的最大数字是 0
        int max_digit = INT32_MIN;
        while (number > 0) {
            int digit = number % 10;
            max_digit = max(max_digit, digit);
            number /= 10;
        }
        return max_digit;
    }

    int maxSum(vector<int>& nums) {
        vector<multiset<int>> mapping(10);
        int max_result = -1;
        for (int i = 0; i < nums.size(); i++) {
            int section = findMaxDigit(nums[i]);
            mapping[section].insert(nums[i]);
            if(mapping[section].size()>1){
                auto it = mapping[section].rbegin();
                max_result = max(max_result, *it + *(++it));
            }
        }
        return max_result;
    }
};

```

- `mymap` 最多只能有 10 个元素，大小固定不需要哈希动态数组就可以
 - 解法中最关键的数据容器是 `multiset` 如果 vector 中使用 set 作为容器，则不允许一个 vector 中有相同的元素，这样就会漏掉数据
- 优化后：

```

class Solution {
public:
    int maxSum(vector<int>& nums) {
        int ans = -1;
        vector<int> max_val(10, INT_MIN);
    }
};

```

```

        for (int v: nums) {
            int max_d = 0;
            for (int x = v; x; x /= 10)
                max_d = max(max_d, x % 10);
            ans = max(ans, v + max_val[max_d]);
            max_val[max_d] = max(max_val[max_d], v);
        }
        return ans;
    }
};

```

其中最关键的两行代码：

```

ans = max(ans, v + max_val[max_d]);
max_val[max_d] = max(max_val[max_d], v);

```

- `max_val` 数组中存储**已知**的每组有相同最高位的数组中最大的数字，如果比原来的数字大，使用 `max_val[max_d] = max(max_val[max_d], v);` 更新
- 用新添加的数字和**已知**的同最高位中最大的一位相加，比之前最大数还大就更新
- 由于只遍历一次添加两个更新，`ans` 会在不同的组之间切换，省去一遍循环
- 计算最大位数的做法在循环的同时取最小值

2342. 数位和相等数对的最大和

(<https://leetcode.cn/problems/max-sum-of-a-pair-with-equal-sum-of-digits/>)

同 [[#2815. 数组中最大数对和|2815. 数组中最大数对和]]，可以使用模拟或者原地求和，最极端的思想就是每一个分组中只需要保留**插入之前的最大的元素**和插入进来的元素比较，比较“最大元素”的**前一个状态**，极值压缩空间

模拟（优化前）：^tfyby1

```

class Solution {
public:
    int getSum(int x, int ans = 0){
        do{
            ans += x % 10;
        } while (x/=10);
        return ans;
    }

    int maximumSum(vector<int>& nums) {
        int max_result = -1;

```

```

unordered_map <int, vector<int>> mapping;
for (int i = 0; i < nums.size(); i++) {
    if(mapping[getSum(nums[i])].size() < 2) {
        mapping[getSum(nums[i])].emplace_back(nums[i]);
    } else if (nums[i] > min(mapping[getSum(nums[i])][0], mapping[getSum(nums[i])][1])) {
        mapping[getSum(nums[i])][0] = nums[i]; // 更新位于“较小的数字”的专用位置——第一
    }
    sort(mapping[getSum(nums[i])].begin(), mapping[getSum(nums[i])].end()); // 最
}
for(auto x : mapping) {
    if(x.second.size() > 1)
        max_result = max(max_result, x.second[0] + x.second[1]);
}
return max_result;
}
};

```

- 这是每组中只保留两个元素：第一大和第二大两个，并不停更新最小的
 - 由于没有容器可以满足允许重复值的同时对插入元素进行排序，且容器大小不变，所以只能用 vector 和 sort 组合，增加了复杂度
- 优化后（访问次数标记法）：

```

class Solution {
public:
    int getSum(int x, int ans = 0) {
        -----
    }

    int maximumSum(vector<int>& nums) {
        int ans = -1;
        unordered_map<int, int> mx;
        for (int num : nums) {
            int s = getSum(num);
            if (mx[s]) { // 如果一个位置被访问过
                ans = max(ans, mx[s] + num); // 计算答案是否被更新
            }
            mx[s] = max(mx[s], num); // 没有被访问过则在该位置更新数据（标记为访问过）
        }
        return ans;
    }
};

```

```

    }
};

```

如果容器是 `vector<int>` (因根据测试数据, 数位和只能是 0~81, 可以创建一个大小为 81 的数组, 省去哈希表的 key 部分), 则可以不用判断是否被访问过 (去掉 if), 因值默认是, 而哈希表不用 if 会导致 `mx[s]` 在访问时创建一个键值对

```

class Solution {
public:
    int getSum(int x,int ans =0){
        -----
    }
    int maximumSum(vector<int>& nums) {
        int n = nums.size();
        vector<int> v(100, INT_MIN);
        int result = -1;
        for (auto num : nums) {
            int sum = getSum(num);
            result = max(result, num + v[sum]);
            v[sum] = max(v[sum], num);
        }
        return result;
    }
};

```

原地删除容器中元素（可能需要补全）

27. 移除元素

(<https://leetcode.cn/problems/remove-element/>)

优化前:

```

class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int count = 0,length = nums.size();
        for (int i = 0; i < nums.size();i++){
            if(nums[i]==val){
                count++;
            }
        }
        return length - count;
    }
};

```

```

        nums.erase(nums.begin() + i--);
    }

    return length - count;
}

};

```

注意：

每循环一次调用一次 `erase` 函数擦除内容，较为费时间，需要注意的是**使用迭代器对容器中部分内容操作**时，尤其是 `erase` 操作，需要在操作后回退一次迭代器，否则会漏掉一个元素[[#剔除字符串中字符|剔除字符串中字符]]

优化分析：

- 每次循环调用一次封装函数，有多次文件 io 跳转，拖慢速度
- 可以使用更新的方式，因返回的数组还是原来的数组，可以将更新的内容从头重新直接赋值给原来的数组
- 使用 `erase` 的过程包括：删除原来数组相应位置的内容+调整数组的大小（这一点在不是动态数组作为底层实现的容器中还可能需重新分配内存（需要复制数据+移动数据））
- 使用重新赋值给原来的数组的方式（是一种双指针方式）在全表扫描完后，剩下的数据是要被剔除的数据，所以可以使用 `resize()` 代替

优化后：

```

class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int current = 0;
        for (int i = 0; i < nums.size(); i++) {
            if (nums[i] != val) {
                nums[current++] = nums[i];
            }
        }
        // nums.erase(nums.begin() + current, nums.end());
        nums.resize(current);
        return current;
    }
};

```

1679. K 和数对的最大数目

(<https://leetcode.cn/problems/max-number-of-k-sum-pairs/>)

使用了[[题型分类#访问次数标记法]]

优化前（算法是错误的）：

```
class Solution {
public:
    int maxOperations(vector<int>& nums, int k) {
        multimap<int, bool> mapping;
        int count = 0;
        for (int i = 0; i < nums.size(); i++) {
            bool flag = true;
            auto it = mapping.find(k - nums[i]);
            if(it!=mapping.end() && it->second==true){
                count++;
                flag = false;
                it->second = flag;
            }
            mapping.insert({nums[i], flag});
        }
        return count;
    }
};
```

- 总体来说满足了各种要求，但错在 `it` 只匹配到第一个键为 `k - nums[i]` 就会立刻终止并返回对应的 `pair`，并不能同时匹配 `it->second==true`，`find` 只接受一个条件，如果需要多个条件可以参考[[用法导向知识#多条件匹配]]
- 不能使用 `while(mapping.find(k - nums[i])!=mapping.end() && mapping.find(k - nums[i])->second==true)`，因 `find` 方法并没有记忆，每一次循环不会从上一次停止的位置开始

换一种思路，用第二个值记录访问次数，（如果还用 `bool` 作为 `value`）

![[Pasted image 20250117122624.png]]

在这种思路下，一个哈希表中 `value` 只能为 **0** 和 **大于 0**，分别表示未匹配和匹配，但因有可能连续多个相同的数字输入导致连续多个 1 输入，`target=3`，再输入多个 2 只会有一个 1 匹配成功（漏了很多，所以值改为 `int` 记录次数） -

```
class Solution {
public:
    int maxOperations(vector<int> &nums, int k) {
        int ans = 0;
        unordered_map<int, int> mapping;
        for (int x : nums) {
```

```

        auto it = mapping.find(k - x);
        if (it != mapping.end() && it->second) {
            it->second--;
            ans++;
        } else {
            mapping[x]++;
        }
    }
    return ans;
}
};

```

记录次数可以避免连续输入相同数导致的匹配丢失，匹配成功就 `--`，由于一个数值只有两种状态，未匹配和匹配（范围也限制在0和大于0），次数记录法同时解决这些问题
 多条件匹配（但会超时），因如果所有数据都挤在一个 key 中，每次 `find_if` 会从 `begin ()` 遍历到 `end()` 时间复杂度为 `O (n*n!)`

```

class Solution {
public:
    int maxOperations(vector<int>& nums, int k) {
        unordered_map<int, bool> mapping;
        int count = 0;
        for (int i = 0; i < nums.size(); i++) {
            bool flag = true;
            int temp = k - nums[i];
            auto it = find_if(mapping.begin(), mapping.end(), [&temp](const pair<int, bool> &p) {
                return p.first == temp && p.second == true; });
            if (it != mapping.end()) {
                count++;
                flag = false;
                it->second = flag;
            }
            mapping.insert({nums[i], flag});
        }
        return count;
    }
};

```

还可以使用 `equal_range` 先匹配键，再匹配值，但要注意由于 [[用法导向知识#equal_range 键匹配|equal_range的返回值设计]]，只能使用有序容器 `multimap`，并且逻辑更复杂，**超时** [[用法导向知识#equal_range 键匹配]]

多余的适配性

数学变换套路

1512. 好数对的数目

(<https://leetcode.cn/problems/number-of-good-pairs/>)

优化前：（省略的是[[固定套路#计算组合数]]）

```
class Solution {
public:
    int combination(int n, int k = 2) { // 计算C(n, k)
        -----省略-----
    }

    int numIdenticalPairs(vector<int>& nums) {
        vector<int> content(100, 0);
        int result = 0;
        for (int i = 0; i < nums.size(); i++) {
            content[nums[i]-1]++;
        }
        for (int i = 0; i <= 99; i++) {
            if (content[i] != 0) {
                result += combination(content[i]);
            }
        }
        return result;
    }
};
```

- 可以发现计算阶乘的指数已经定为 **2**，所以 $C(n, 2) = n(n-1)/2$ 固定，且等于公差为一的等差数列的前 n 项和，所以可以将两个循环整合为一个，每次 `+=` 时，将要加的数字加上一即可，不需要借助 `combination` 函数
- 这种只是利用了题目的测试集限制，`vector<int> content(100, 0);` 也并不好，在这里只作为一种算法通过数学转换优化提供的一种思路，可以通过其他方式实现优化后：

```

class Solution {
public:
    int numIdenticalPairs(vector<int>& nums) {
        int ans = 0;
        unordered_map<int, int> cnt;
        for (int x : nums) {
            ans += cnt[x]++;
        }
        return ans;
    }
};

```

固定的输入输出

1108. IP 地址无效化

(<https://leetcode.cn/problems/defanging-an-ip-address/>)

题目中输入输出都按照固定格式，所以可以在检索、查找，匹配这三步上省去开销

```

char *defangIPaddr(char *address){
    int a, b, c, d;
    char *str = malloc((strlen(address) + 10) * sizeof(char));
    sscanf(address, "%d.%d.%d.%d", &a, &b, &c, &d);
    sprintf(str, "%d[.]%d[.]%d[.]%d", a, b, c, d);
    return str;
}

```

- `sscanf` 是一种控制，格式化输入的方法，代码中只会将所有 `%d` 部分存入 a, b, c, d 中，但却要求 address 以 `"%d.%d.%d.%d"` 展示
- 在“替换”（这里已经不是替换操作了，而是创建一个新字符串）输出时，也是用 `sprintf` 保证输出按照 `"%d[.]%d[.]%d[.]%d"` 格式，但调用的是 a, b, c, d 中内容
- 最终存储在多了 9 个 char 类型的指向 char 类型的指针位置（多一个安全裕量）
- 所有的变量都使用 `*` 直接操作相应地址，省去复制时间

2446. 判断两个事件是否存在冲突

(<https://leetcode.cn/problems/determine-if-two-events-have-conflict/>)

冲突判定思路都是判断两段时间的开始和结束时间

优化前：

```

class Solution {
public:
    bool haveConflict(vector<string>& event1, vector<string>& event2) {
        int start_time1 = stoi(event1[0].substr(0, 2)) * 60 + stoi(event1[0].substr(3, 2))
        int end_time1 = stoi(event1[1].substr(0, 2)) * 60 + stoi(event1[1].substr(3, 2))
        int start_time2 = stoi(event2[0].substr(0, 2)) * 60 + stoi(event2[0].substr(3, 2))
        int end_time2 = stoi(event2[1].substr(0, 2)) * 60 + stoi(event2[1].substr(3, 2))
        if ((start_time1 <= end_time2) && (start_time2 <= end_time1))
            return true;
        else
            return false;
    }
};

```

通过简化 if-else，直接化成一行 `return`，并省略变量存储而直接判断可以简化为：

```

class Solution {
public:
    bool haveConflict(vector<string>& event1, vector<string>& event2) {
        return ((stoi(event1[0].substr(0, 2)) * 60 + stoi(event1[0].substr(3, 2))) <= stoi(event2[0].substr(0, 2)) * 60 + stoi(event2[0].substr(3, 2))) && ((stoi(event1[1].substr(0, 2)) * 60 + stoi(event1[1].substr(3, 2))) <= stoi(event2[1].substr(0, 2)) * 60 + stoi(event2[1].substr(3, 2)));
    }
};

```

优化分析：

- 但可以发现输入永远都是两组，一组两个时间，每个时间 5 个字符
- 时间的比较大小并不需要转换为能够计算的 int 类型并且比较，**字符串也可以比较大小**，并且使用字典顺序比较的字符串，是从头开始比较，相同则向下轮，这就是一种时间比较的逻辑，所以可以直接使用字符串比较方式来比较时间

```

class Solution {
public:
    bool haveConflict(vector<string> &event1, vector<string> &event2) {
        return event1[0] <= event2[1] && event1[1] >= event2[0];
    }
};

```

对于比较的不是时间而是更大的、其他规则的输入，前面的公式计算法适配性更强，但固定输入，还是取巧为好

![[Pasted image 20241220093812.png|虽然性能并没有多大提升（汗）]]

从特定题目中总结

前缀和

2559. 统计范围内的元音字符串数

(<https://leetcode.cn/problems/count-vowel-strings-in-ranges/>)

优化前：

```
class Solution {
public:
    vector<char> vowel = {'a', 'e', 'i', 'o', 'u'};
    bool is_correct (string x){
        if(find(vowel.begin(),vowel.end(),x[0])!=vowel.end()&&find(vowel.begin(),vowel.end(),x[x.size()-1])!=vowel.end())
            return true;
        }
        return false;
    }
    vector<int> vowelStrings(vector<string>& words, vector<vector<int>>& queries) {
        vector<int> result;
        vector<int> sum(words.size());
        int count = 0;
        for (int i = 0; i < words.size();i++){
            if(is_correct(words[i])){
                ++count;
            }
            sum[i] = count;
        }
        for (int j = 0; j < queries.size();j++){
            result.emplace_back(sum[queries[j][1]] - sum[queries[j][0]] + is_correct(words[queries[j][1]]));
        }
        return result;
    }
};
```

- 在数据量比较大时，对每个元素调用函数实现计算会浪费一些时间在调用过程，直接在算法内实现能提高性能

- 前缀和一般设置大小为**数组大小+1**，因这样方便下标作为实际序号，也免去了 `num[i]=nums[i-1]+nums[i]` 带来的**溢出问题**，第一个元素设置值为 0
- 能用下标访问就不要用 `emplace_back` 等构造方法
- `sum[queries[j][1]] - sum[queries[j][0]] + is_correct(words[queries[j][0]])` 这行代码中下标看似是常数级别，但原理是**迭代访问**，嵌套访问次数太多性能开销较大
- 小技巧：需要访问一个容器最后一个元素时，不要 `container[container.size()-1]`，使用 `.back()` 方法自动返回最后一个元素
- 在类似哈希表的容器中，使用 `find` 查找一个元素是否存在其中（无论是容器的 `.find` 方法还是 `algorithm` 库中 `find`）都没有 `.count` 方法更快
 - 如果是 `algorithm` 中 `find`，从 `begin` 到 `end`，平均复杂度为 n
 - 使用对应容器的 `find`，红黑树（底层实现）一直都是 $\log n$ ，哈希表平均 1，哈希表结构只有在所有元素都在一个 `hashbucket` 中时才会退化为 n

优化后：

```
class Solution {
public:
    vector<int> vowelStrings(vector<string>& words, vector<vector<int>>& queries) {
        unordered_set<char> vowels = {'a', 'e', 'i', 'u', 'o'};
        int n = words.size();
        vector<int> sum(n + 1);
        for(int i = 0; i < n; i++) {
            if(vowels.count(words[i][0]) && vowels.count(words[i].back()))
                sum[i + 1] = 1 + sum[i];
            else
                sum[i + 1] = sum[i];
        }
        vector<int> result(queries.size());
        for (int i = 0; i < queries.size(); i++) {
            result[i] = sum[queries[i][1]+1] - sum[queries[i][0]];
        }
        return result;
    }
};
```

合适的数据结构

位运算作为容器

1470. 重新排列数组

(<https://leetcode.cn/problems/shuffle-the-array/>)

由于测试集大小原因，int 类型有很多位没有用上一直都是 0，利用起来！

空间复杂度为 $O(1)$ 的两种解法（比较难想到）

1930. 长度为 3 的不同回文子序列

(<https://leetcode.cn/problems/unique-length-3-palindromic-subsequences/>)

ABA 结构，A 与 B 互相联系，枚举中间的 B 是解法

超时写法：（这是枚举两边 A 而不是 B）

- 通过计算并集，对每一个固定的字符枚举并集中元素相当于枚举 A 了
- 复杂度较高，因计算交集会有多个冗余统计，复杂的数据结构也会拖慢速度

```
class Solution {
public:
    int countPalindromicSubsequence(string s) {
        int n = s.length(), count = 0;
        set<char> temp1 = {}, temp2 = {};
        // 每遍历一个需要就加一个字符串看看是否重复，使用set维护
        vector<set<char>> pre(n), after(n);
        //pre[i-1]记录s[i]左边有什么字符，用set存储无重复且排序
        //after记录右边同理
        vector<char> common; // 最后pre[i-1]和after[i+1]的交集
        unordered_set<string> only; // 已经匹配过的字符串
        string temp3;
        for (int i = 0, j = n - 1; i < n, j >= 0; i++, j--) {
            temp1.insert(s[i]);
            pre[i] = temp1;
            temp2.insert(s[j]);
            after[j] = temp2;
        }
        for (int k = 1; k < n - 1; k++) {
            set_intersection(pre[k - 1].begin(), pre[k - 1].end(), after[k + 1].begin(),
                             after[k + 1].end(), common.begin());
            if (common.size() != 0) {
                for (auto c : common) {
                    string str = {c, s[k], c}; // 由于使用set存储的是char类型，需要组合成string
                    if (only.insert(str).second == true) {
                        count++; // 插入成功，表示这个字符串没有被匹配过
                    }
                }
            }
        }
        return count;
    }
};
```

```

        }
    }
    common.clear();// 下一个字符的左右两边不同，所以common清零
}
}
return count;
}
};

```

乍一看 n 复杂度还不错，但 `set_intersection` 函数会遍历 `pre` 和 `after`，复杂度为 $O(26*26)$ ，再放入 `for` 中，复杂度为 $O(26*26*n)$ 时间已经爆了，更何况每个 `for` 中复杂逻辑 [[用法导向知识#求两个容器的交并补集]]

简化思路：

```

class Solution {
public:
    int countPalindromicSubsequence(string s) {
        int n = s.size();
        int res = 0;
        vector<int> pre(n), suf(n); // pre表示
        for (int i = 0, j = n - 1; i < n, j >= 0; ++i, --j) {
            // 前缀 s[0..i-1] 包含的字符种类
            pre[i] = (i ? pre[i-1] : 0) | (1 << (s[i] - 'a'));
            // 后缀 s[i+1..n-1] 包含的字符种类
            suf[j] = (j != n - 1 ? suf[j+1] : 0) | (1 << (s[j] - 'a'));
        }
        // 每种中间字符的回文子序列状态数组
        vector<int> ans(26);
        for (int i = 1; i < n - 1; ++i) {
            ans[s[i] - 'a'] |= (pre[i-1] & suf[i+1]);
        }
        // 更新答案
        for (int i = 0; i < 26; ++i) {
            res += __builtin_popcount(ans[i]);
        }
        return res;
    }
};

```

```

    }
};

```

- 其中按位或运算在判断是否已经匹配中很关键
 - 原解法中使用 `unordered_set` 存储前后缀字符串原因是
 - 寻找并集 `set_intersection` 需要排好序的容器作为参数
 - `char` 类型变量不能拼接**，
 - 不能使用 `static_cast<string>(char)` 将 `char` 转化为 `string`
 - 不能有重复的运算，因题目要求重复不纳入计算
 - 使用位运算
 - 一个 bit 上是 0/1 表示是否有这个数字，保证不重复
 - `1 << (s[i] - 'a')` 遍历到的字符是多少就会让对应位置的数字位变为 1 标记这个数字已经出现过了同时保证了有序
 - int 有 32 位，字母 26 个，不存在拼接问题
 - 可以容纳，并且不需要拼接，对应数字位上 `!=0` 可以得出有哪些字符
- 完美解决所有数据结构的要求
- `ans[s[i]-'a'] |= (pre[i-1] & suf[i+1]);` 按位与运算得到（用二进制表示的字符交集）结果
- 把结果转化为数字累加到答案上，`_builtin_popcount` 用来统计二进制数字中 1 的个数，有 `_builtin_popcountll`，`_builtin_popcountll` 变体计算不同数据类型的二进制 1 个数。[[用法导向知识#统计二进制数字中 1 / 0 的个数]]

红黑树和哈希表

1394. 找出数组中幸运数

(<https://leetcode.cn/problems/find-lucky-integer-in-an-array/>)

优化前：

```

class Solution {
public:
    int findLucky(vector<int>& arr) {
        unordered_map<int, int> hashTable;
        for(int x : arr){
            hashTable[x]++;
        }
        int result = -1;
        for(pair<int,int> temp : hashTable){

```

```

        if(temp.first == temp.second){
            result = max(temp.first, result);
        }
    }
    return result;
}
};

```

- 在找哈希表中内容时，可以直接使用 `hashTable[key]` 操作而不必使用 `find()` 函数，`hashTable[x]` 将新增键值对和对键值对操作，搜索三个功能为一体
- 对于有些题中"如果没有找到，则....."，可以将最初需要返回的值初始化为没有找到情况下需要返回的值，省去一步 `else` 的工作

优化分析：

- 优化前用到两次遍历，不能一次原因是不能在统计个数时比较是否成为幸运数字并且找到最大的幸运数字（如果强行实现会导致一次循环逻辑很复杂，也较难实现）
- 如下

```

for(int x : arr){
    hashTable[x]++;
    if(hashTable[x] == x){
        // 这里添加判断代码
    }
}

```

- 由于是否为幸运数字需要判断数组中还有没有了，也就是说使用乱序的表判断是否为幸运数字一定要等遍历完一遍数组

```

class Solution {
public:
    int findLucky(vector<int>& arr) {
        sort(begin(arr), end(arr));
        int a=1;
        for(int i=arr.size()-1; i>0; i--){
            if(arr[i]==arr[i-1]){
                a++;
            }
            else{
                if(a==arr[i]) return arr[i];
            }
        }
        return -1;
    }
};

```

```

        a=1;
    }

    }

    if(arr[0]==a) return arr[0];

    return -1;

}

};

```

- 如果表示有序的，则可通过判断一个**暂时为幸运数字的下一位**是否与上一位相同来判断他是不是，可以遍历的同时添加计数器，记录最大值的同时判断幸运数字
- 题目要求返回最大的幸运数字，排序后**从后往前遍历找到的第一个幸运数字就是需要返回的数字**，从前往后则不能直接返回
- 由于遍历不到第一个数字，需要在循环后添加一个第一个数字的判断，保证倒序遍历完了才触发这个特殊的情况
- 最后第一个数字也不符合，返回 `-1`
- 节省了一个哈希表内存大小，只遍历一次，排序一次 ($O(\log n + n)$)

2418. 按身高排序

(<https://leetcode.cn/problems/sort-the-people/>)

优化前：

```

class Solution {
public:
    vector<string> sortPeople(vector<string>& names, vector<int>& heights) {
        map<int, string> mapping;
        vector<string> results(heights.size());
        for (int i = 0; i < heights.size(); i++) {
            mapping[heights[i]] = names[i];
        }
        for (map<int, string>::reverse_iterator it = mapping.rbegin(); it != mapping.rend(); it++) {
            results.emplace_back(it->second);
        }
        return results;
    }
};

```

- map 使用红黑树维护，其中迭代器是[[用法导向知识#迭代器类型|前向迭代器]]，不支持，**每一次**查询和增删改的复杂度是 $O(\log n)$ ，unordered_map 使用哈希表维护，**每一次**查询和增删

改的复杂度是 $O(1)$

- 算法中使用 map 作为存储，提供[[用法导向知识#迭代器类型|双向迭代器]]，所以使用反向迭代器将内容放入 results 中，是的每一次遍历的都需要 $O(\log n)$ ，总共 n 次，所以第二个 for 循环需要 $O(n \log n)$ ，这时可以想到哈希表每次查找更快，但哈希表中元素无序
- 可以想到插入后对所有数据排序一次（sort 复杂度 $O(n \log n)$ ），然后使用哈希表解决所有查询操作

优化后：

```
class Solution {
public:
    vector<string> sortPeople(vector<string>& names, vector<int>& heights) {
        vector<int> index(heights.size()); // 索引数组
        vector<string> result; // 答案数组
        iota(index.begin(), index.end(), 0); // 初始化索引数组
        sort(index.begin(), index.end(), [&](int i, int j) { return heights[i] > heights[j]; });
        for(int i=0; i<heights.size(); i++) result.emplace_back(names[index[i]]); // 通过排
        return result; // 返回最后的答案
    }
};
```

- 可以知道，其实 heights 中数据是多少根本不重要，重要的是他们能够比大小，并且唯一，一个递增或者递减的序列也可以完成这两个功能，并且一个从零开始的递增序列还能充当下标使用
- 发现这一点那么原数据就没有用了，为节省空间可以直接复写
- 所以可以使用人为创建索引方法，将 index 中内容重写成下标
- 其中 sort 第三个参数可以使用 `sort(index.begin(), index.end(), ::greater());` greater

1512. 好数对的数目

(<https://leetcode.cn/problems/number-of-good-pairs/>)

使用哈希表分类容器元素

分析过程在[[固定套路#使用哈希表辅助维护并分类、操作]]中可见

不重复枚举

跳过逻辑上的不可能

15. 三数之和

(<https://leetcode.cn/problems/3sum/>)

优化前:

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        vector<vector<int>> ans;
        int n = nums.size();
        for (int i = 0; i < n - 2; i++) {
            int x = nums[i];
            if (i && x == nums[i - 1]) continue;
            int left = i + 1, right = n - 1;
            while (left < right) {
                int s = nums[right] + nums[left] + x;
                if (s > 0) right--;
                else if (s < 0) left++;
                else {
                    ans.emplace_back(vector<int>(nums[i], nums[left], nums[right]));
                    for (left++; left < right && nums[left] == nums[left - 1]; left++);
                    for (right--; left < right && nums[right] == nums[right + 1]; right--);
                }
            }
        }
        return ans;
    }
};
```

使用[[题型分类#相向双指针]]思想:

- `If(i && x == nums[i - 1]) continue;` 将重复值跳过比较, 由于使用 `sort` 后相同的元素一定相邻, 所以重复的情况可以用 `nums[i] == nums[i - 1]` 比较, 也防止了溢出

![[Pasted image 20250215222848.png|350]]

![[Pasted image 20250215221822.png]]

- 如果 `[current]t+[left]+[right] ? 0` 通过[[题型分类#双指针法#167. 两数之和 II - 输入有序数组]]可知比较的结果决定了 `left` 或者 `right` 的移动

- 外层循环推动 `current`, 内层推动 `left` 和 `right` 的靠拢

- 找到一个答案三元组添加 `ans` 中后, 还需要继续判断 `left` 和 `right` 之间有没有 `answer`

```
for (left++;left < right && nums[left] == nums[left - 1]; left++);
for (right--;left < right && nums[right] == nums[right + 1];right--);
// 这里使用两个for将left和right跳转到没有重复相邻元素的位置,进入下一循环
```

- 其中找到下一个没有重复元素的位置时, `left < right` 必须有且在 `nums[left] == nums[left - 1]` 前面, 没有的话会出现因 `left++` 导致 `left==right` 已经出现但外层循环还没停止

优化分析:

- 左指针 `i+1`, 右指针 `n-1`, 两者向中间并拢可以得到当前 `i` 的答案, 这样复杂度为 $O(n^2)$, 所以需要优化, 将逻辑上一定不成立的元素跳过
- 如果 `nums[current] > 0`, 那么 `nums[left]` 和 `nums[right]` 一定也 > 0 , 没必要判断相加, 所以在拿到 `current` 后立刻判断 (1)

![[Pasted image 20250215223318.png|离current最近的2个数 > 0]]

![[Pasted image 20250215223425.png|离current最远的2个数 < 0]]

两种情况 (2) (3) 不用再比较了, 同样在拿到 `current` 后立刻判断

优化后:

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        ranges::sort(nums);
        vector<vector<int>> ans;
        int n = nums.size();
        for (int i = 0; i < n - 2; i++) {
            if (nums[i] > 0) break; // (1)
            int x = nums[i];
            if (i && x == nums[i - 1]) continue;
            if (x + nums[i + 1] + nums[i + 2] > 0) break; // (2)
            if (x + nums[n - 2] + nums[n - 1] < 0) continue; // (3)
            int left = i + 1, right = n - 1;
            while (left < right) {
                int s = nums[right] + nums[left] + x;
                if (s > 0) right--;
                else if (s < 0) left++;
                else {
                    ans.emplace_back(vector<int>{nums[i], nums[left], nums[right]});
                    for (left++;left < right && nums[left] == nums[left - 1]; left++);
                    for (right--;left < right && nums[right] == nums[right + 1];right--);
                }
            }
        }
        return ans;
    }
};
```

```
        }  
    }  
}  
return ans;  
}  
};
```